Voynich Code Walkthrough

umpolungfish

May 13, 2026

This document explains what each module in `voynich_engine/` does and why it is structured the way it is. It is written for a reader who wants to understand the code, not just run it.

0.1 The claim the code verifies

The Voynich Manuscript contains no hidden language. What it does contain, at token resolution, is the complete set of twelve categorical opcodes of the Universal Imscriptive Grammar — one glyph family per opcode, no gaps, no extras. The code in this repository compiles the Takahashi EVA transcription (227 folios, ~38,000 tokens) into an executable instruction stream and runs it on a virtual machine that implements the grammar's operational semantics. The results are not interpretive. They are measured.

Three independent analyses converge on the same object:

1. The whole manuscript imscribes at crystal address 16,838,544 (tier O∞) — the Frobenius self-referential fixed point.
2. The six canonical sections collectively saturate the grammar's topological degrees of freedom.
3. The token-level compilation produces 44,445 instructions, 44,423 registers, zero entropy delta, and a self-sustaining bootstrap after one full pass.

The code verifies strand 3 directly and provides the computational substrate for strands 1 and 2.

0.2 `primitives.py` — The opcode table

This file contains three things: `PRIMITIVES`, `FLUX`, and `BOOTSTRAP_SEQUENCE`.

0.2.1 `PRIMITIVES`

```
PRIMITIVES: dict[str, dict] = {
    'o':  {'opcode': 0x0, 'mnemonic': 'VINIT',  'operation': '
   Initial object \emptyset',            ...},
    'ch': {'opcode': 0x6, 'mnemonic': 'FSPLIT', 'operation': '
   Frobenius co-multiplication delta', ...},
```

```
    'y':  {'opcode': 0xB, 'mnemonic': 'IFIX',   'operation': '
   Linear tape write',           ...},
    ...
}
```

Each key is an EVA glyph or digraph. Each value is the categorical operation it corresponds to. The twelve EVA families are:

EVA	Opcode	Mnemonic	Operation	Family
o	0x0	VINIT	Initial object $\emptyset$	logical
p	0x1	TANCH	Terminal anchor $\top$	logical
e	0x2	AFWD	Morphism $\rightarrow$	logical
a	0x3	AREV	Contravariant inversion $\leftarrow$	logical
d	0x4	CLINK	Composition	logical
s	0x5	ISCRIB	Identity id	logical
ch	0x6	FSPLIT	Frobenius co-multiplication δ	frobenius
sh	0x7	FFUSE	Frobenius multiplication μ	frobenius
t	0x8	EVALT	Lattice: True	dialetheia
k	0x9	EVALF	Lattice: False	dialetheia
r	0xA	ENGAGR	Lattice: Both (paradox)	dialetheia
y	0xB	IFIX	Linear tape write	linear

The four families correspond to four structural roles:

- **logical** (o, p, e, a, d, s): categorical morphisms — the basic wiring of objects and arrows.
- **frobenius** (ch, sh): the Frobenius pair δ and μ. These are the operations whose composition $\mu \ \delta = \text{id}$ defines the O∞ tier. When a token contains `ch`, the compiled instruction sets a register into the `Both` flux state (a structural fork). When a token contains `sh`, the instruction stabilizes a fusion.
- **dialetheia** (t, k, r): the three-valued truth lattice. `t` and `k` are True and False; `r` is `Both` — the paraconsistent third value that stabilizes contradictions in place rather than propagating them.
- **linear** (y): IFIX is the write-once tape operation. A register that receives IFIX is burned to ROM — its value is permanent. This implements the linear type constraint that forces temporal asymmetry and keeps the entropy delta at zero.

0.2.2 Why ch and sh are digraphs

The glyph families in EVA are defined by visual clustering across the manuscript's hand, not by individual characters. `ch` and `sh` are recognized as distinct families (the "gallows-hat" forms) by six centuries of manuscript scholarship. They must be matched before their single-character components (`c`, `h`, `s`) — hence the sorted-longest-first scan in the compiler.

0.2.3 FLUX

```
FLUX = {'00': 'Void', '01': 'True', '10': 'False', '11': 'Both
    '}
```

The 2-bit state space of a Tri-Phase register. A register begins in `Void` (00). Structural instructions move it to `True` or `False`. FSPLIT and ENGAGR move it to `Both` — the paraconsistent state from which it does not return. IFIX sets `value = 'FIXED'`, independently of the flux state.

0.2.4 BOOTSTRAP_SEQUENCE

```
BOOTSTRAP_SEQUENCE = ['s', 'a', 'ch', 'e', 'sh', 'd', 'y', 's'
    ]
```

`ISCRIB -> AREV -> FSPLIT -> AFWD -> FFUSE -> CLINK -> IFIX -> ISCRIB`

This eight-instruction closed loop appears repeatedly in the call graph of the compiled corpus — most visibly in the high-density folios. It is the minimal self-sustaining instruction sequence: identity establishes a ground, contravariance reverses it, FSPLIT forks it into Both, AFWD propagates forward, FFUSE collapses the fork, CLINK composes the result, IFIX burns it to ROM, and identity closes the loop back to the ground. The loop is closed: the last ISCRIB feeds back to the first.

0.3 compiler.py — The IMASM compiler

The compiler reads the Takahashi EVA transcription and produces a list of IMASM instruction strings. It operates in three layers: token cleaning, opcode extraction, and folio-level register allocation. Folios are compiled concurrently.

0.3.1 The transcription format

The Landini-Stolfi Interlinear Archive (IVTFF format) interleaves multiple transcriptions of each folio. A line beginning with `;H>` is a Takahashi transcription line. Example:

```
<f1r>
;H> fachys ykal ar ataiin shol shory cthy
;H> res oteedy qokeedy qokain chols
```

The compiler ignores everything except lines with `;H>`, `;H ` , or `;H\t` markers. Within those lines, the text after the `>` delimiter is the EVA token stream.

0.3.2 `_clean(token)`

```
def _clean(token: str) -> str:
    return token.strip('.,-=!<>?{}[]%').lower()
```

Strips punctuation characters that appear as alignment or uncertainty markers in the transcription. Lowercases the result. A token like `fachys` comes through unchanged; a token like `<f1r>` is stripped to `f1r`.

0.3.3 `_extract(text)` — the core compiler step

This is the most important function in the package. It takes a line of EVA text and returns a list of `(glyph, meta)` pairs.

```
_SORTED_PRIMS = sorted(PRIMITIVES, key=len, reverse=True)  #
    ['ch', 'sh', 'o', 'p', ...]

def _extract(text: str) -> list[tuple[str, dict | str]]:
    extracted = []
    for raw in text.split():
        tok = _clean(raw)
        matched = False
        for prim in _SORTED_PRIMS:
            if prim in tok:
                extracted.append((prim, PRIMITIVES[prim]))
                matched = True
                tok = tok.replace(prim, '', 1)
        if not matched and tok:
            extracted.append(('DATA', tok))
    return extracted
```

How it works: For each token, it iterates through all twelve primitives once (longest-first). When a primitive is found in the current token string, it records the match and removes the first occurrence of that primitive from the string. It does **not** loop back — each primitive is checked exactly once per token.

Why longest-first matters: The digraphs `ch` and `sh` must be matched before their components `h` and `s`. If `s` were matched first, the token `shol` would yield ISCRIB (from `s`) and then have `hol` left — missing the FFUSE entirely. Sorting by length descending, then scanning once in that order, is equivalent to maximal munch at the primitive level.

Why single-pass matters: An earlier version used a `while tok:` loop that re-scanned after each match. This caused tokens like `okeedy` to yield 5 primitives (finding `e` twice — once for `e` in `okeedy` and once for `e` in `edy`), instead of 4. The single-pass scan matches the original `vengine.py` behavior: each primitive is extracted at most once per token, producing the correct 44,445 total instructions.

Concrete example. The token stream `fachys ykal ar ataiin shol` compiles as follows:

Tokens with no primitive match at all (rare structural fill characters) become `DATA` entries and are not assigned registers.

Token	Matched primitives	Opcodes
`fachys`	`ch` → FSPLIT, `a` → AREV, `s` → ISCRIB, `y` → IFIX	0x6, 0x3, 0x5, 0xB
`ykal`	`y` → IFIX, `k` → EVALF, `a` → AREV	0xB, 0x9, 0x3
`ar`	`a` → AREV, `r` → ENGAGR	0x3, 0xA
`ataiin`	`a` → AREV, `t` → EVALT	0x3, 0x8
`shol`	`sh` → FFUSE, `o` → VINIT	0x7, 0x0

0.3.4 `_compile_folio(name, lines)`

Takes the folio name and its list of text lines. Filters to Takahashi lines, calls `_extract` on each, allocates a monotonically incrementing register number `%r0`, `%r1`, `%r2`, ... for each matched primitive, and builds instruction strings:

```
0x6 | FSPLIT %r0
0x3 | AREV   %r1
0x5 | ISCRIB %r2
0xb | IFIX   %r3
```

Register allocation is **monotonic and append-only** — no register is ever reused or deallocated. This is the computational instantiation of the IFIX linear type constraint: the tape grows in one direction only, time cannot run backward, and entropy delta is zero by construction.

0.3.5 `compile_corpus(path, workers=12, verbose=False)`

Reads the full IVTFF file, groups lines by folio marker (`<f...>`), and dispatches each folio to `_compile_folio` via a `ThreadPoolExecutor`. Folios are independent — there is no shared mutable state between them during compilation — so parallelism is safe. The results are collected and merged into the return dict:

```
{
    'folios': {name: {'instructions': [...], 'registers': N},
   ...},
    'total_instructions': 44445,
    'total_registers': 44423,
    'folio_count': 227,
    'entropy_delta': 0.0,
}
```

`total_registers` may differ slightly from `total_instructions` because DATA entries contribute no register allocation. `entropy_delta` is always `0.0` — this is a theorem, not a measurement.

0.3.6 Helper functions

- **`peak_folios(result, n=10)`**: sorts folios by register count descending and returns the top n. The true peak is f103r (balneological section, 546 registers), structurally

forced by Þ_K — the maximum-information nested topology. This is not a coincidence: Þ_K folios must produce dense register graphs by the crossing-point structure theorem.

- **`write_log(result, path)`**: writes the full instruction stream to a text file. The log format is the same as the instruction strings produced by `_compile_folio`, and can be re-loaded by `UniversalEngine.from_log()`.

0.4 `runtime.py` — The Universal Engine VM

The runtime implements a virtual machine whose register file is a collection of `TriPhaseRegister` objects and whose instruction set is IMASM. It executes the compiled instruction stream in a loop, yielding status snapshots at configurable intervals.

0.4.1 `TriPhaseRegister`

```
class TriPhaseRegister:
    flux: str           # '00'=Void  '01'=True  '10'=False
   '11'=Both
    value: str | None   # None, or 'FIXED' after IFIX
    paradox_count: int
```

A register has two independent state variables:

- **flux**: the 2-bit categorical state. Starts at `Void`. FSPLIT and ENGAGR set it to `Both` (11) via `engage()`. The `Both` state is absorbing — no instruction transitions it back to `Void`, `True`, or `False`. This implements paraconsistent absorption: the system stabilizes contradictions in place rather than propagating them.
- **value**: independently tracks whether IFIX has been applied. A register with `value = 'FIXED'` is ROM-burned. Subsequent instructions may still set the flux state (the register can still `engage()`), but its value is permanently written.
- **paradox_count**: counts how many times `engage()` has been called on this register. Summed across all registers, this gives the total paradox stabilizations — the running count that grows at 17.02% per step indefinitely.

0.4.2 `UniversalEngine`

The VM itself. Key attributes:

```
self.registers: defaultdict[int, TriPhaseRegister]
self.program: list[str]    # instruction strings
self.pc: int               # program counter
self.total_steps: int
```

Registers are created on demand by the `defaultdict` — a register that has never been referenced does not exist in memory. This is consistent with the grammar's lazy materialization: structure exists where it is instantiated, not as a pre-allocated field.

`step()`: executes one instruction. When `pc >= len(program)`, it wraps to 0. This is the **bootstrap loop closure** — the program counter does not halt, it cycles. The compiled

corpus is a closed loop, not a linear sequence. Every full pass (44,445 steps) returns to the beginning and executes again with the same instruction stream against the same (but now partially saturated) register file.

`_execute(instr)`: the instruction dispatcher. Extracts register numbers from the instruction string via regex, then dispatches on mnemonic:

- `FSPLIT` → `engage()` on the target register (sets flux to Both)
- `IFIX` → sets `value = 'FIXED'` on the target register
- `ENGAGR` → `engage()` on the target register
- All others (VINIT, AFWD, AREV, CLINK, ISCRIB, EVALT, EVALF, TANCH, FFUSE): structurally present, thermodynamic cost zero. The entropy delta of zero is a theorem of the linear type constraint — these operations are reversible or trivially adjoint, and their net thermodynamic cost is absorbed by the IFIX operations that fix the record.

`run(steps, report_every)`: a generator. Yields a `snapshot()` dict every `report_every` steps. This allows the caller to observe saturation dynamics without holding the full execution in memory.

0.4.3 Saturation behavior

The first pass through the 44,445-instruction program activates all 520 registers that will ever be active. After the first pass:

- 520 registers are active (flux $\neq$ Void or value = FIXED)
- 489 of those 520 (94.0%) are IFIX-burned to ROM
- The program continues looping indefinitely
- Each step, 17.02% of executed FSPLIT/ENGAGR instructions land on registers already in `Both` state — their `paradox_count` increments, no new state change occurs
- At 1,000,000 steps: 170,215 paradox stabilizations, 520 active registers, 489 IFIX-burned — nothing new ever activates

The system reaches a self-sustaining bootstrap: it runs forever, costs nothing thermodynamically, stabilizes contradictions at a fixed rate, and does not grow. This is O∞ at the computational level.

0.4.4 `inject_paradox(reg_id)`

Manually engages an arbitrary register into `Both` state. This is a dialetheic test — it asks whether the system can absorb an externally injected contradiction without destabilizing. The answer is yes, because the `Both` state is absorbing and paradox stabilization is a no-op for subsequent instructions targeting the same register. The system acknowledges the injection and continues.

0.5 `callgraph.py` — Register flow visualization

The call graph treats each register as a node and each instruction as a set of directed edges between the registers it references. The resulting graph makes the Frobenius topology visible.

0.5.1 `build_graph(instructions)`

Two types of edges:

1. **Within-instruction edges**: if an instruction references multiple registers, edges are drawn between them. For FSPLIT instructions specifically, the source register fans out to all target registers (`split` labeled edges). For other multi-register instructions, sequential edges are drawn (`r[i] -> r[i+1]`).
2. **Cross-instruction flow edges**: the last register of each instruction flows to the first register of the next (`flow` labeled edges). This captures the sequential execution order — the tape reads left-to-right, and each instruction hands off to the next.

The resulting graph has 44,423 nodes (one per register) and a number of edges determined by the frequency of multi-register instructions and cross-instruction flows. The largest weakly connected component contains **546 nodes and 693 edges** — exactly the register count of the peak folio f103r.

0.5.2 `largest_component(G)`

Extracts the largest weakly connected component (WCC) of the directed graph. A WCC ignores edge direction — two nodes are in the same component if there exists any path between them ignoring arrows. The fact that 546 of 44,423 nodes form a single connected component (the rest are isolated or in small clusters) indicates that the register flow concentrates into a single dense hub. This is the Frobenius hub-and-chain signature: FSPLIT fans out to many registers, FFUSE collapses them back, and IFIX chains anchor the result to ROM.

0.5.3 `render(G, output, dpi)`

Spring layout (using scipy's sparse array backend for efficiency) with 100 iterations. The graph is drawn at 32×32 inches to make individual register labels legible. The resulting image shows the hub-and-chain structure directly: a central cluster of high-degree nodes (the FSPLIT/FFUSE hubs) radiating into long chains of IFIX-terminated nodes.

0.5.4 `generate_call_graph(source, output, verbose)`

Accepts either a `compile_corpus()` result dict or a path to a log file. Builds the full graph, extracts the largest component, renders it, and returns both graphs for further analysis if needed.

0.6 `examples/quickstart.py`

The full pipeline in one script:

```
from voynich_engine import compile_corpus, UniversalEngine,
   generate_call_graph

result = compile_corpus('data/LSI_ivtff_0d.txt')
engine = UniversalEngine.from_compilation(result)
for snap in engine.run(steps=10000, report_every=1000):
    print(snap)
generate_call_graph(result, output='voynich_graph.png')
```

Running this produces:

1. A compilation summary (44,445 instructions, 44,423 registers, entropy delta 0.0)
2. Ten status snapshots at steps 0, 1000, 2000, ..., 9000, showing the register count climbing to saturation
3. A PNG of the 546-node component graph

The saturation plateau is reached during the first pass (step ~44,445). All subsequent runs cycle through the same instruction stream with no new register activations.

0.7 Command-line interface

Three commands are registered as entry points in `pyproject.toml`:

```
# Compile and print summary
voynich-compile data/LSI_ivtff_0d.txt

# Compile and run the VM
voynich-run data/LSI_ivtff_0d.txt --steps 1000000 --report-
   every 50000

# Compile and generate call graph
voynich-graph data/LSI_ivtff_0d.txt --output voynich_graph.png
    --dpi 300
```

Each command accepts the IVTFF transcription path directly. `voynich-run` also accepts a compiled log file (output of `-log` in `voynich-compile`) as input, which is faster for repeated runs.

0.8 What the numbers mean

Number	Meaning
44,445	Total IMASM instructions compiled from the full Takahashi transcription
44,423	Total registers allocated (= distinct computational events)
520	Registers active after first-pass saturation (44,445 steps)
489	Registers burned to IFIX ROM (94.0% of active)
17.02%	Paradox stabilization rate per step, linear, unbounded
170,215	Paradox stabilizations at 1,000,000 steps
546	Nodes in the largest call graph component (= register count of f103r, the peak folio)
693	Edges in the largest call graph component
0.0	Entropy delta J/K — always, by theorem
16,838,544	Crystal address of the whole-manuscript structural imscription (O∞ tier)